

プロジェクト仕様書 - 位置情報常時取得アプリ

プロジェクト概要

このプロジェクトは、ユーザーの位置情報を常時取得し、Supabaseに保存するiOS対応のモバイルアプリケーションです。

取得した位置情報は、別途作成されるERPシステムで表示・管理されます。

プロジェクト目的

- アプリをダウンロードしたユーザーの位置情報を常時取得
- バックグラウンドでも位置情報の取得を継続
- 取得した位置情報をリアルタイムでSupabaseに保存
- ERPシステムからの位置情報参照を可能にする

技術スタック

- フロントエンド: React Native (Expo)
- バックエンド: Supabase
- 位置情報取得: expo-location, expo-task-manager
- ストレージ: @react-native-async-storage/async-storage
- 使用言語: JavaScript (TypeScript ではない)
- 対応プラットフォーム: iOS

アプリケーション機能要件

1. 位置情報取得機能

1.1 権限管理

- 前景位置情報権限: アプリ使用中の位置情報取得
- 背景位置情報権限: バックグラウンドでの位置情報取得

- 権限リクエストダイアログの表示
- 権限が許可されない場合のエラーハンドリング

1.2 位置情報取得設定

- **精度:** 高精度（GPS使用）
- **時間間隔:** 60秒ごと
- **距離間隔:** 10メートル移動ごと
- **バックグラウンド動作:** 継続的に取得
- **自動停止:** 無効（常に取得し続ける）

1.3 取得する位置データ

- 緯度（latitude）
- 経度（longitude）
- 高度（altitude）
- 精度（accuracy）
- 速度（speed）
- 方向（heading）
- タイムスタンプ（timestamp）

2. ユーザー識別機能

2.1 ユーザーID管理

- 初回起動時にユーザーIDの入力を求める
- ユーザーIDをローカルストレージに保存
- 位置情報とユーザーIDを紐付けて保存

3. データ保存機能

3.1 Supabase連携

- 取得した位置情報を自動的にSupabaseに送信
- オフライン時のデータ保持（オプション）
- 保存エラー時のリトライ処理

4. 追跡制御機能

4.1 追跡開始・停止

- 追跡開始ボタン
- 追跡停止ボタン
- 現在の追跡状態表示（追跡中/停止中）

4.2 ステータス表示

- 追跡状態のビジュアル表示
- 最終取得時刻の表示
- エラーメッセージの表示

画面設計

ホーム画面（メイン画面）

レイアウト要素

1. ヘッダー部分

- アプリタイトル: "位置情報追跡アプリ"
- ユーザーID表示

2. ステータス表示部分

- 追跡状態インジケータ（追跡中は緑色、停止中はグレー）
- 追跡状態テキスト（"追跡中" / "停止中"）
- 最終位置情報取得時刻
- 現在の位置情報（緯度・経度）

3. コントロール部分

- ユーザーID入力フィールド（初回のみ / 変更時）
- 追跡開始ボタン（大きく目立つボタン）
- 追跡停止ボタン

4. 情報表示部分

- エラーメッセージ表示エリア
- 取得回数カウンター（オプション）

データベース設計

Supabaseテーブル設計

locationsテーブル

位置情報を保存するメインテーブル

カラム名	データ型	制約	説明
id	uuid	PRIMARY KEY, DEFAULT uuid_generate_v4()	自動生成ID
user_id	text	NOT NULL	ユーザー識別ID
latitude	double precision	NOT NULL	緯度
longitude	double precision	NOT NULL	経度
altitude	double precision	NULL	高度（メートル）
accuracy	double precision	NULL	位置精度（メートル）
speed	double precision	NULL	速度（m/s）
heading	double precision	NULL	方向（度）
timestamp	timestamptz	NOT NULL	位置情報取得日時
created_at	timestamptz	DEFAULT now()	レコード作成日時

インデックス

- user_id にインデックス（検索性能向上）
- timestamp にインデックス（時系列検索用）
- user_id, timestamp の複合インデックス

RLSポリシー (Row Level Security)

- INSERT: 認証不要（匿名での位置情報登録を許可）
- SELECT: ERP側のサービスロールキーでのみアクセス可能

```
project-root/
├── docs/                                # ドキュメント
│   ├── プロジェクト仕様書.md
│   ├── 開発ルール.md
│   ├── GitHubルール.md
│   └── AI用プロンプト/
│       ├── copilot-instructions.md
│       ├── AGENTS.md
│       ├── claude.md
│       └── Gemini.md
├── src/
│   ├── features/                        # 機能ごとにまとめる
│   │   └── counter/                    # カウンター機能（デモ）
│   │       ├── components/            # UI部品
│   │       ├── screens/               # 画面
│   │       ├── hooks/                 # カスタムフック
│   │       └── constants.js           # 定数
│   ├── shared/                          # 共通で使うもの
│   │   ├── components/                # 汎用コンポーネント
│   │   ├── hooks/                     # 汎用フック
│   │   ├── utils/                     # ユーティリティ関数
│   │   ├── constants/                 # 全体の定数
│   │   └── contexts/                  # Context API
│   ├── navigation/                     # ナビゲーション
│   ├── services/                       # 共通サービス
│   │   ├── supabase/
│   │   │   └── client.js
│   │   └── gas/
│   │       └── gasApi.js
│   └── assets/                          # 静的ファイル
├── supabase/                           # Supabase設定
├── gas/                                # Google Apps Script
├── .claude/                             # Claude Code設定
│   └── CLAUDE.md
├── .env.example
├── .gitignore
└── app.json
```

```
├─ package.json
└─ README.md
```

API 設計

Supabase API

位置情報登録API

- テーブル: locations
- 操作: INSERT
- エンドポイント: Supabase REST API (自動生成)
- リクエストデータ:

```
{
  "user_id": "string",
  "latitude": number,
  "longitude": number,
  "altitude": number | null,
  "accuracy": number | null,
  "speed": number | null,
  "heading": number | null,
  "timestamp": "ISO8601 timestamp"
}
```

バックグラウンドタスク

LOCATION_TASK

- タスク名: background-location-task
- トリガー: 位置情報の更新 (60秒間隔 or 10m移動)
- 処理内容:
 - i. 位置情報データを受信
 - ii. ユーザーIDを取得
 - iii. Supabaseに位置情報を保存
 - iv. エラーハンドリング

ディレクトリ構造

```
project-root/
├── docs/                                     # ドキュメント
│   ├── プロジェクト仕様書.md
│   ├── 開発ルール.md
│   ├── GitHubルール.md
│   └── AI用プロンプト/
│       ├── copilot-instructions.md
│       ├── AGENTS.md
│       ├── claude.md
│       └── Gemini.md
├── src/
│   ├── features/                           # 機能ごとにまとめる
│   │   ├── location/                      # 位置情報追跡機能
│   │   │   ├── components/               # UI部品
│   │   │   ├── screens/                 # 画面
│   │   │   │   └── LocationTrackerScreen.js
│   │   │   ├── hooks/                   # カスタムフック
│   │   │   │   └── useLocationTracking.js
│   │   │   ├── services/                # サービス
│   │   │   │   └── locationService.js
│   │   └── constants.js                 # 定数
│   ├── shared/                           # 共通で使うもの
│   │   ├── components/                  # 汎用コンポーネント
│   │   ├── hooks/                      # 汎用フック
│   │   ├── utils/                      # ユーティリティ関数
│   │   ├── constants/                  # 全体の定数
│   │   └── contexts/                   # Context API
│   ├── navigation/                      # ナビゲーション
│   ├── services/                        # 共通サービス
│   │   └── supabase/
│   │       └── client.js
│   └── assets/                          # 静的ファイル
├── supabase/                             # Supabase設定
│   └── migrations/                      # マイグレーションファイル
├── .env.example
└── .gitignore
```

- └─ app.json
- └─ package.json
- └─ README.md

開発の進め方

フェーズ1: 環境構築

1. Supabase プロジェクトを作成
2. locations テーブルを作成
3. 環境変数を設定 (.env ファイル)
4. 必要なパッケージのインストール

フェーズ2: 位置情報取得機能実装

1. 位置情報サービスの実装
2. バックグラウンドタスクの設定
3. 権限管理の実装

フェーズ3: UI実装

1. メイン画面の作成
2. ユーザーID入力機能
3. 追跡制御ボタンの実装
4. ステータス表示の実装

フェーズ4: データ保存機能実装

1. Supabase連携
2. エラーハンドリング
3. オフライン対応 (オプション)

フェーズ5: テスト

1. 位置情報取得テスト
2. バックグラウンド動作テスト
3. データ保存テスト

4. 実機テスト (iOS)

セットアップ手順

1. リポジトリのクローン

```
git clone [repository-url]
cd Actual_Length_Position_Information_app
```

2. 依存関係のインストール

```
npm install
```

必要なパッケージ:

- expo-location
- expo-task-manager
- @react-native-async-storage/async-storage
- @supabase/supabase-js
- @react-navigation/native
- @react-navigation/native-stack

3. Supabaseプロジェクトの設定

3.1 Supabaseプロジェクト作成

1. <https://supabase.com> でプロジェクトを作成
2. プロジェクトURLとAnon Keyを取得

3.2 locationsテーブル作成

SQLエディタで以下を実行:

```

-- locations テーブル作成
CREATE TABLE locations (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id TEXT NOT NULL,
  latitude DOUBLE PRECISION NOT NULL,
  longitude DOUBLE PRECISION NOT NULL,
  altitude DOUBLE PRECISION,
  accuracy DOUBLE PRECISION,
  speed DOUBLE PRECISION,
  heading DOUBLE PRECISION,
  timestamp TIMESTAMPTZ NOT NULL,
  created_at TIMESTAMPTZ DEFAULT now()
);

-- インデックス作成
CREATE INDEX idx_locations_user_id ON locations(user_id);
CREATE INDEX idx_locations_timestamp ON locations(timestamp);
CREATE INDEX idx_locations_user_timestamp ON locations(user_id, timestamp);

-- RLSポリシー設定
ALTER TABLE locations ENABLE ROW LEVEL SECURITY;

-- INSERT許可 (匿名アクセス可能)
CREATE POLICY "Allow anonymous insert" ON locations
  FOR INSERT
  WITH CHECK (true);

-- SELECT許可 (認証済みユーザーのみ)
CREATE POLICY "Allow authenticated select" ON locations
  FOR SELECT
  USING (auth.role() = 'authenticated');

```

4. 環境変数の設定

```
cp .env.example .env
```

.env ファイルを編集:

```

EXPO_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=your-anon-key

```

5. アプリの起動

```
npm start
```

6. iOS実機でのテスト

```
npm run ios
```

または、Expo Goアプリでスキャン

iOS固有の設定

app.json設定内容

- `bundleIdentifier` : アプリの一意識別子
- `NSLocationAlwaysAndWhenInUseUsageDescription` : 常時位置情報取得の理由
- `NSLocationWhenInUseUsageDescription` : 使用中の位置情報取得の理由
- `UIBackgroundModes` : バックグラウンド実行モード (location)

TestFlightでの配布 (オプション)

1. Expo EAS Buildでビルド
2. App Store Connectにアップロード
3. TestFlightで配布

非機能要件

パフォーマンス

- 位置情報取得間隔: 60秒または10m移動
- バックグラウンド動作: 継続的
- データ送信: リアルタイム

セキュリティ

- ユーザーIDは暗号化せずに保存（識別用途のみ）
- 位置情報は暗号化された通信で送信（HTTPS）
- Supabase RLSでアクセス制御

可用性

- オフライン時の動作: データはローカルに一時保存（オプション）
- エラー時のリトライ: 自動リトライ機能

バッテリー消費

- 高精度GPSを使用するためバッテリー消費が大きい
- ユーザーに事前に通知が必要

制約事項

技術的制約

- iOS専用（Android対応は今回対象外）
- バックグラウンド動作はiOSの制限に従う
- 位置情報権限が必須

ビジネス的制約

- ERPシステムでの表示機能は別プロジェクト
- 今回はデータ取得・保存のみ実装
- ユーザー認証機能は実装しない（ユーザーIDのみ）

プライバシー

- App Store申請時に位置情報利用目的の明記が必要
- ユーザーの同意が必須
- プライバシーポリシーの整備が必要

今後の拡張予定

将来的に追加する可能性のある機能

- Android対応
- オフライン時のデータ同期
- 位置情報履歴の表示
- 取得間隔の設定変更
- バッテリー節約モード
- プッシュ通知機能

ライセンス

MIT License